

SYSTEM DESIGN INTERVIEW

Design WhatsApp

1:1 + group chat at planet scale, end-to-end encrypted, with presence and attachments

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Connection Model
- 06** · Message Protocol & Lifecycle
- 07** · Inbox / Outbox Model
- 08** · End-to-End Encryption (Signal Protocol)
- 09** · Group Chat & Fan-out
- 10** · Presence & Typing Indicators
- 11** · Media Attachments

12 · Offline Delivery & Multi-Device

13 · Scalability & Distribution

14 · Failure Modes & Recovery

15 · Design Trade-offs

16 · Interview Playbook

01

Problem Statement

Requirements and scope

Design **WhatsApp**: a planet-scale messaging service supporting 1:1 and group chats, end-to-end encrypted, with presence (online / last-seen / typing), media attachments, and reliable delivery across mobile + desktop. *Voice and video calling are explicitly out of scope.*

Clarifying Questions

Question	Answer
Users / DAU?	~2B registered, ~500M DAU
Message volume?	~100B messages/day across all chats
Media volume?	~2B media messages/day, ~50KB average
Encryption?	End-to-end (Signal protocol); server stores ciphertext only
Group size?	Historically ~256, now up to ~1024 members
Multi-device?	Yes — phone + up to 4 linked desktop / web devices
Retention?	Server stores until ack from all recipient devices, then evict
Voice/video?	Out of scope (separate WebRTC / SFU pipeline)

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
1:1 chat	Send / receive text + media; ordering preserved per pair
Group chat	Up to ~1024 members; admin controls; member add/remove
Receipts	Sent (server ack), Delivered (device ack), Read (user opens chat)
Presence	Online / offline / last-seen / typing — privacy controlled
Attachments	Images, video, docs up to ~100MB; thumbnails auto-generated
Multi-device	Same account on phone + 4 linked devices, ciphertext per device
Offline delivery	Queue messages until recipient connects; deliver on reconnect
E2E encryption	Signal protocol (X3DH + Double Ratchet); sender keys for groups

Non-Functional Requirements

Requirement	Target
Availability	99.99% — messaging is the critical path
Latency (online → online)	p50 < 200 ms, p99 < 1 s end-to-end
Latency (push to offline)	p99 < 5 s via APNs / FCM wakeup
Throughput	~1.16M msgs/sec average, 5–10× peak (~10M/sec)
Durability	No message loss once server-acked; per-device idempotency
Privacy	Server sees only ciphertext + minimal envelope metadata
Scale	~100M concurrent persistent connections globally

03

Capacity Estimation

Math for planet scale

Traffic Estimation

- Registered users: **~2B**; DAU: **~500M**
- Daily messages: **~100B/day** across all chats
- Average msg/sec: $100B / 86,400 \approx 1.16M \text{ msgs/sec}$
- Peak factor: **5–10×** (New Year's Eve, World Cup) → **~6–12M msgs/sec peak**
- Concurrent connections: **~100M globally** (subset of DAU online at any moment)

Storage Estimation

- Ciphertext envelope: **~1KB** per text message average (header + payload + MAC)
- Active queue (in-flight + offline) $\approx 100B \times 1KB = 100 \text{ TB/day}$ of inbox writes — but evicted on ack
- Steady-state inbox storage: **~5–10%** of users have undelivered queue at any time → **~10–20 TB hot**
- Media: $2B \text{ media messages/day} \times 50KB \text{ avg} = \sim 100 \text{ PB/day}$ of media — stored in S3-class object store
- Media is referenced by URL inside the encrypted envelope; bytes themselves are ciphertext

Connection / Edge Estimation

- Concurrent persistent TLS sessions: **~100M**
- Per edge server budget: **~50K connections/server** (tuned C10K-style sockets)
- Edge fleet: $100M / 50K = \sim 2,000 \text{ edge servers}$ globally; round to 3–5K with HA + region overhead
- Heartbeat: **30–60 sec** (presence + NAT keepalive) → **~1.6–3.3M heartbeats/sec**

KEY

Memorize These Numbers

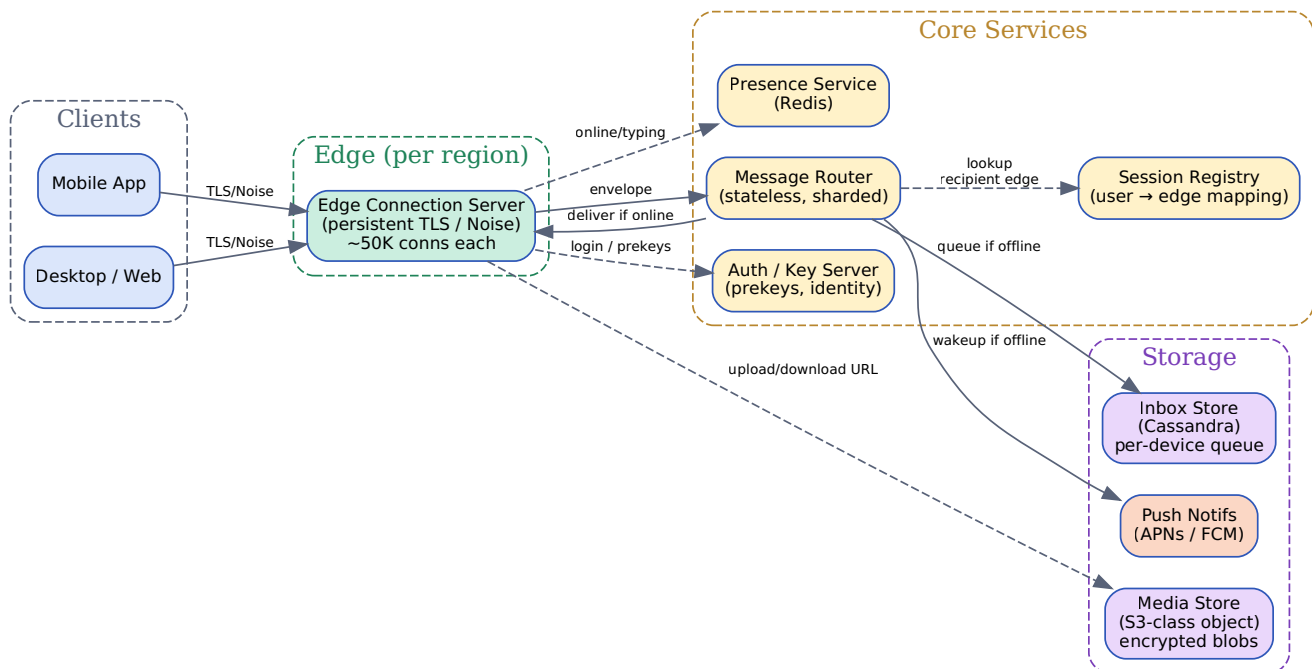
100B msgs/day · **1.16M msgs/sec avg** · **~10M msgs/sec peak** · **100M concurrent connections** ·
~50K conn/edge · **100 PB/day media.**

04

High-Level Architecture

System overview

Clients open a single persistent TLS connection (XMPP-derived, WhatsApp uses a binary variant called Noise Protocol on top of TCP/TLS) to the nearest **Edge Connection Server**. The edge server authenticates the device, then relays messages to a stateless **Message Router** that looks up the recipient's home edge (via a presence / session registry) and pushes the ciphertext there. If the recipient is offline, the router writes the envelope to the recipient's **Inbox Store** (Cassandra) and triggers a push notification (APNs / FCM) to wake the device.



Online → online path goes router-to-router; offline path lands in Cassandra and is fetched on reconnect.

Architecture Highlights

- **Edge Connection Server:** terminates TLS / Noise, owns the persistent socket, ~50K connections/box
- **Message Router:** stateless; consults Session Registry to find the recipient's edge; falls back to inbox store on miss
- **Session Registry:** in-memory KV (Redis / etcd) mapping `user_device` → `edge_node`; updated on connect/disconnect
- **Presence Service:** Redis cluster keyed by `user_id`; tracks online state, typing, last-seen with TTL
- **Auth / Key Server:** stores user identity keys + bundles of one-time prekeys for X3DH initial handshake
- **Inbox Store:** Cassandra; per-device row of pending envelopes; messages evicted after device ack
- **Media Store:** S3-class object store; client-side encrypted blobs; CDN edges for download

- **Push Notifications:** APNs (iOS) and FCM (Android) wake the app to fetch from inbox when offline

05

Connection Model

Persistent TLS, sharded edges, heartbeats

Every online device holds a single long-lived TCP+TLS connection to its nearest edge server. The protocol is binary, framed, and multiplexed: a single connection carries chat messages, receipts, presence updates, and key-exchange traffic.

Why Persistent Connections

- **Push without polling:** server can deliver instantly without the client asking
- **Battery / data:** single TCP keepalive cheaper than HTTP polling at 1.16M msg/sec scale
- **Ordering:** in-order frames per connection simplify per-pair ordering guarantees
- **NAT traversal:** outbound long-lived TCP punches NAT and stays open via heartbeats

Heartbeats and Liveness

- Client sends heartbeat every **30–60 seconds** (NAT timeout windows are typically 90 s+)
- Edge marks connection dead after **2 missed heartbeats** and removes from Session Registry
- Presence is derived from the existence of a healthy connection, not a separate ping
- Mobile radios prefer batched heartbeats — Android jobs / iOS background fetch align with OS schedule

Sharding the Edge Fleet

- **Geographic routing:** client resolves `chat.whatsapp.net` via GeoDNS to nearest PoP
- **Inside a region:** L4 load balancer hashes by `user_id` for sticky reconnect
- **~50K connections/server:** tuned with epoll + non-blocking I/O (Erlang/BEAM at WhatsApp historically)
- **~2,000 edge servers** for 100M concurrent connections; ~3–5K with replication / spare capacity

INSIGHT

Why Erlang at WhatsApp

WhatsApp famously ran on Erlang/BEAM because the runtime is built around millions of lightweight processes, preemptive scheduling, and supervision trees — exactly the shape of a problem where each connection is an independent state machine that must survive partial failures of its neighbours.

06

Message Protocol & Lifecycle

Send, ack, deliver, read

A message walks through four states visible to the user: **sent** (server received), **delivered** (recipient device received), **read** (recipient opened the chat). Each transition is an explicit ack frame; the sender's UI updates the checkmarks accordingly.



Each ack hop is independent. Server ack only proves durability; device ack proves delivery; read receipt is opt-in.

Wire Envelope (server-visible)

```
// Server only sees this – payload is opaque ciphertext.
Envelope {
  msg_id:      uuid_v4,          // client-generated; idempotency key
  from:        user_device_id,   // sender device
  to:          user_device_id,   // one envelope PER recipient device
  ts_client:   int64_ms,         // sender clock
  ts_server:   int64_ms,         // assigned at edge ingress
  type:        enum{TEXT, MEDIA, RECEIPT, TYPING, KEY_EXCHANGE},
  ratchet_hdr: bytes,            // Double Ratchet header (DH key + counters)
  ciphertext:  bytes,            // AES-256-GCM payload (text or media descriptor)
  mac:         bytes,            // HMAC over header + ciphertext
}
```

Lifecycle Steps

1. Sender encrypts payload per recipient device (Double Ratchet) and writes envelopes to its outbound queue
2. Sender's edge receives envelope, assigns `ts_server` + `msg_id` dedup, returns **server ack** (single check)
3. Router looks up each recipient device in Session Registry: online → push to recipient edge, offline → write to Cassandra inbox
4. Recipient edge delivers envelope, recipient device decrypts, sends **delivered ack** back through the chain (double check)
5. When recipient opens the chat (and read receipts are on), device sends a **read ack** (blue ticks)
6. After all recipient devices have acked, router evicts the envelope from inbox storage

07

Inbox / Outbox Model

Per-device queues in Cassandra

WhatsApp does not store your message history on the server. Instead, the server is a relay with a transient queue: each **device** has a per-device inbox of pending envelopes, and once every recipient device has acked, the server evicts the envelope. The authoritative chat history lives on the client (and in encrypted backups to iCloud / Google Drive).

Cassandra Inbox Schema

```
-- Per-device pending envelopes. One row per (recipient_device, msg).
-- Partition key spreads load; clustering key gives in-order pull.
CREATE TABLE inbox (
  recipient_device_id uuid,          -- partition key: one device's queue
  enq_ts               timeuuid,     -- clustering key: server enqueue time
  msg_id              uuid,          -- idempotency key
  sender_device_id    uuid,
  envelope             blob,         -- ciphertext + ratchet header
  attempts            int,           -- delivery retry count
  expires_at          timestamp,     -- TTL ~30 days max
  PRIMARY KEY ((recipient_device_id), enq_ts)
) WITH CLUSTERING ORDER BY (enq_ts ASC)
   AND default_time_to_live = 2592000; -- 30 days hard cap

-- Idempotency dedup: avoid double-enqueue if router retries.
CREATE TABLE inbox_dedup (
  recipient_device_id uuid,
  msg_id              uuid,
  PRIMARY KEY ((recipient_device_id, msg_id))
) WITH default_time_to_live = 604800; -- 7 days
```

Why Cassandra

- **Write-heavy:** 1.16M envelope writes/sec ingress is exactly Cassandra's sweet spot
- **Partition by recipient_device_id:** all of one device's pending messages live on one set of replicas — single-partition pull on reconnect
- **TTL native:** Cassandra evicts expired rows automatically; no separate cleanup job
- **Tunable consistency:** QUORUM writes for durability, ONE reads for latency
- **Ring expansion:** add nodes online; consistent hashing redistributes only 1/N keys

Eviction Policy

Trigger	Action
Recipient device acks delivery	DELETE row from inbox immediately
All recipient devices in chat acked	Sender's outbox row also evicted
TTL = 30 days expired	Cassandra tombstones row; message lost (rare)
User uninstalls / re-registers	All inbox rows for that device purged

TIP

Server is a Relay, Not an Archive

This design choice is what lets WhatsApp claim it cannot read your messages: even if compelled, the server has only the brief transit ciphertext, not the history. Long-term chat backups are encrypted client-side with a key the user controls.

08

End-to-End Encryption (Signal Protocol)

X3DH, Double Ratchet, Sender Keys

WhatsApp uses the **Signal Protocol: X3DH** (Extended Triple Diffie-Hellman) for the initial key agreement, and the **Double Ratchet** for ongoing message keys with forward secrecy and post-compromise security. Group chats use a derived construction called **Sender Keys** for efficiency.

X3DH — First Contact

- Each user publishes to the Auth/Key server: **identity key (IK)**, **signed prekey (SPK)**, and a batch of **one-time prekeys (OPK)**
- Alice fetches Bob's bundle (IK_B, SPK_B, one OPK_B) — the OPK is consumed and never reused
- Alice computes 3 (or 4) Diffie-Hellman shared secrets across her keys and Bob's; concatenates → KDF → **shared root key**
- From the root key both sides derive the initial Double Ratchet state — no round-trip needed for Alice's first message

Double Ratchet — Ongoing Messages

Each message has its own message key, derived by stepping a **symmetric ratchet** (KDF chain). When the peer replies, a fresh **DH ratchet** step mixes a new ephemeral DH key into the root, giving forward secrecy (past messages stay safe if the current key leaks) and post-compromise security (future messages recover after key leak).

```
# Simplified Double Ratchet send step.
def ratchet_encrypt(state, plaintext, associated_data):
    # 1) Symmetric-ratchet the chain key to get this message's key.
    state.CKs, mk = kdf_ck(state.CKs)          # CKs -> (next CKs, message key)
    header = Header(
        dh    = state.DHs.public,
        pn    = state.PN,                      # prev chain length
        n     = state.Ns,                      # this msg index
    )
    state.Ns += 1
    ciphertext = aead_encrypt(
        key    = mk,
        plain  = plaintext,
        ad     = associated_data + serialize(header),
    )
    return header, ciphertext

def ratchet_rcv(state, header, ciphertext, ad):
    if header.dh != state.DHr:
```

```

# Peer ratcheted forward: do a DH step and reset chains.
skip_message_keys(state, header.pn)
state.DHr = header.dh
state.RK, state.CKr = kdf_rk(state.RK, dh(state.DHs, state.DHr))
state.DHs = generate_dh()
state.RK, state.CKs = kdf_rk(state.RK, dh(state.DHs, state.DHr))
skip_message_keys(state, header.n)
state.CKr, mk = kdf_ck(state.CKr)
return aead_decrypt(mk, ciphertext, ad + serialize(header))

```

Group Chat — Sender Keys

Running a separate Double Ratchet to each of 1023 other group members for every message is wasteful — the sender would encrypt and the server would store 1023 ciphertexts per message. Instead, the sender generates a **sender key** (a symmetric chain key) and distributes it once to each member via their pairwise Signal session. Subsequent group messages are encrypted **once** with the sender key and the server fans out the ciphertext.

Property	Pairwise Double Ratchet	Sender Key (Group)
Encrypt cost (sender)	$O(N)$ per msg	$O(1)$ per msg, $O(N)$ once at setup
Server storage / msg	N ciphertexts	1 ciphertext, fanned out at delivery
Forward secrecy	Per-message via DH ratchet	Per-message via symmetric ratchet only
Member added/removed	Trivial	Rotate sender key + redistribute

WARNING

Server Sees Metadata, Not Content

E2E encryption hides payload but not who-talks-to-whom-and-when. The router still sees sender id, recipient id, timestamps, and message size. Threat-modelling WhatsApp must distinguish content privacy (strong) from metadata privacy (weak).

09

Group Chat & Fan-out

Client-side vs server-side fan-out

There are two fan-out designs for group chat. **Client-side fan-out**: the sender encrypts and uploads N copies, one per recipient device. **Server-side fan-out**: the sender uploads one ciphertext + a member list and the server duplicates it to N inboxes. WhatsApp uses a **hybrid**: the sender encrypts *once* with the sender key and the server fans out the single ciphertext to all member edges / inboxes.

Comparison

Approach	Sender cost	Server cost	Privacy	Used for
Client-side per-recipient	Encrypt N times, upload N	Storage = N	Server sees ciphertext + recipient list	Initial sender-key distribution
Server-side fan-out (plain)	Encrypt once, upload 1	Multicast to N inboxes	Server would need plaintext — INCOMPATIBLE WITH E2E	Not used for content
Sender Key (hybrid, what WhatsApp uses)	Encrypt once with chain key	Multicast 1 ciphertext to N recipient inboxes	Server stays oblivious to content	All ongoing group messages

Sender-Key Fan-out Pseudocode

```
# Sender side (per group, once group is established):
def send_group_message(group_id, plaintext):
    sk_state = sender_keys[group_id]      # symmetric chain key
    sk_state.chain, mk = kdf_ck(sk_state.chain)
    ciphertext = aead_encrypt(mk, plaintext, ad=group_id || sk_state.idx)
    sk_state.idx += 1
    envelope = {
        'group_id': group_id,
        'sender': my_device_id,
        'sk_idx': sk_state.idx,
        'ciphertext': ciphertext,
    }
    upload_to_edge(envelope)              # ONE upload

# Server side (in Message Router):
def route_group(envelope):
    members = group_members(envelope.group_id)  # cached, ~1024 max
```

```

for device_id in members:
    if device_id == envelope.sender:      # don't echo to self
        continue
    edge = session_registry.get(device_id)
    if edge:
        edge.push(device_id, envelope)    # online path
    else:
        inbox.append(device_id, envelope)  # offline path
        push_notif.wake(device_id)

```

Membership Changes

- **Add member:** existing members each pairwise-encrypt the current sender key for the new member
- **Remove member:** all remaining members *rotate* sender keys so the removed member can't decrypt future traffic
- **Admin actions** are themselves messages routed through the same protocol — no privileged server path

INSIGHT

Why ~1024 is the Practical Cap

Each new member triggers $O(N)$ pairwise sender-key distributions; each removed member forces $O(N)$ rotation. The cost is bearable up to $\sim 10^3$ members; beyond that people typically use Channels (broadcast) which has different threat-model and drops per-recipient encryption.

10

Presence & Typing Indicators

Online, last-seen, typing — privacy-aware

Sources of Presence Truth

- **Online:** the user has at least one healthy edge connection (heartbeat fresh)
- **Last-seen:** timestamp of the most recent disconnect or background event
- **Typing:** sender broadcasts a TYPING control frame; recipient edge fans out

Storage & Propagation

- Presence Service: **Redis cluster** keyed by user_id, fields = {online, last_seen, typing_in}, TTL ~120 s
- Updates pushed only to **contacts in active conversations** (not globally) — N to a few, not N to millions
- Typing TTL ~10 s so a stale TYPING auto-clears if the sender disconnects mid-keystroke
- Heartbeat refresh extends online TTL; disconnect or missed beats → set last_seen + flip online=false

Privacy Modes

Setting	Visible to	Server behavior
Last-seen: Everyone	All contacts	Broadcast last_seen to subscribers
Last-seen: My Contacts	Contacts only	Filter subscribers by contact list
Last-seen: Nobody	No one	Suppress presence broadcast entirely
Read receipts: off	Sender sees only ✓ ✓ (delivered)	Drop READ ack on the floor at recipient edge
Typing: off (silent typing not exposed in UI)	—	Still sent on wire if sender's UI emits

TIP

Don't Broadcast to the World

The naive design — push 'Alice is online' to all 500 of Alice's contacts every time she opens the app — costs $500M \times 100 \text{ events/day} = 50B \text{ presence msgs/day}$. WhatsApp only pushes presence to contacts who currently have Alice's chat open, cutting fan-out by 100×.

11

Media Attachments

Encrypted blobs in object store

Sending a 50 KB image through the message envelope path would balloon the inbox store and choke the router. Instead, media takes a **side channel**: the client encrypts the file with a fresh symmetric key, uploads ciphertext to a **Media Store** (S3-class), and the chat envelope carries only the URL + the symmetric key. Recipients pull from the CDN edge.

Upload Flow

1. Client generates random **media_key** (32 bytes) and encrypts file with AES-CBC + HMAC
2. Client requests **pre-signed PUT URL** from media-upload service (one-shot, expires in minutes)
3. Client PUTs ciphertext directly to object store — bytes never touch the message router
4. Client computes content hash; checks server-side hash for **dedup** (popular forwards = same blob)
5. Client constructs media descriptor {url, media_key, sha256, mime, size, thumbnail}
6. Descriptor is **encrypted inside the normal Signal envelope** and sent to recipients

Download Flow

1. Recipient decrypts envelope with Double Ratchet → recovers media descriptor + media_key
2. Recipient GETs the ciphertext blob from CDN (cache hit on viral forwards)
3. Recipient verifies HMAC, decrypts with media_key, and renders
4. Server never sees plaintext — even though the media bytes are stored on Meta-controlled S3

Storage Economics

Tier	Use	Notes
Hot (S3 Standard / Cassandra inbox)	Last 30 days	Heaviest read; CDN-fronted
Warm (S3 IA)	30–180 days	Cheaper; access latency tens of ms
Cold (Glacier)	180 days – years	Forward chains; rarely re-fetched
Eviction	Reference count = 0	All recipient devices acked + downloaded → delete

KEY

100 PB/day, but Dedup Saves the Day

2B media msgs/day $\times$ 50KB = ~100 PB/day raw. Forward dedup (same SHA on blob) typically reduces unique storage by 3–5 $\times$ because a fraction of media is heavily re-shared. Net steady-state media corpus growth is materially smaller than the ingress rate suggests.

12

Offline Delivery & Multi-Device

Reconnect, replay, retry

Reconnect Flow

1. Device opens TLS to nearest edge; presents auth token + device fingerprint
2. Edge writes (user_device → edge_node) into Session Registry
3. Edge issues a single Cassandra query: `SELECT * FROM inbox WHERE recipient_device_id = ? ORDER BY enq_ts`
4. Edge streams pending envelopes to device in order; device acks each
5. On ack receipt, edge issues DELETE for the row (or buffers and batches)
6. Device is now caught up; subsequent messages flow online via the router

Offline Retry Policy

Stage	Action	Retry
Recipient online	Edge push	TCP retransmit; if disconnect → fall through
Recipient offline (recent)	Inbox write + APNs/FCM wakeup	Re-wake every 1 min, capped at 3 attempts
Recipient offline (long)	Inbox holds with TTL = 30 days	No additional pushes; pulled on reconnect
TTL exceeded (~30 days)	Cassandra evicts row	Message lost; sender keeps single check (sent only)
Device permanently gone (uninstall)	Server detects via key rotation	Drop pending envelopes; sender notified

Multi-Device

- Each linked device has its own **identity key + Signal session** with every contact
- A single chat to Bob means the sender encrypts *once per Bob device* (phone + 2 desktops = 3 envelopes)
- Linked desktops historically required the phone to be online; modern WhatsApp uses an independent multi-device protocol
- Device-add: new device fetches the chat history via an authenticated bootstrap from a primary device, not from the server

INSIGHT

Per-Device, Not Per-User

Every encryption decision in WhatsApp's modern protocol is keyed on device, not user. A user with phone + iPad + 2 desktops is 4 endpoints; ack semantics, sender keys, and inbox rows are all per-device.

13

Scalability & Distribution

Sharding, regions, and the registry

Horizontal Scaling Cheatsheet

Tier	Scaling axis	Why it works
Edge Connection Server	Add servers, GeoDNS routes new conns	Stateful per connection but stateless across users
Message Router	Stateless, scale on CPU	All state in Session Registry / Cassandra
Session Registry	Shard by user_id	O(1) lookup; replicate across AZs
Cassandra Inbox	Add ring nodes; partition by recipient_device_id	Hot-key risk minimal — each device has its own queue
Presence (Redis)	Cluster slots by user_id	Soft-state, can be rebuilt from heartbeats
Media Store	Object store auto-scales	Pay-by-use, CDN absorbs read

Geographic Distribution

- **Edge PoPs** in every major region; clients connect to nearest via GeoDNS / Anycast
- **Cassandra ring** typically multi-region with local-quorum writes; cross-region replication async
- **Cross-region routing:** if Alice (Europe) messages Bob (Asia), Alice's edge → her region's router → Bob's region's router → Bob's edge
- **Failover:** if a region edge is unhealthy, GeoDNS pulls it out of rotation; client retries to next-nearest PoP

Hot Spots and Hot Groups

- Celebrity broadcast → 1024 members all online → 1024 fan-out per message; bounded by group cap
- Trending forward (viral image) → media CDN handles bursts; envelope rate not affected because content is by-reference
- Marketing flood / spam → rate limit at edge by sender, not recipient (cheaper)

14

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Edge server crash	All ~50K connections drop simultaneously	Heartbeat loss + LB health	Clients reconnect to peer edge via GeoDNS; Session Registry updates on reconnect
Session Registry partition	Router can't find recipient edge — falls through to inbox	Registry RTT spike	Treat as 'offline' (write inbox + push); registry self-heals
Cassandra ring partition	Some inbox writes fail at QUORUM	Coordinator timeout	Hinted handoff buffers writes; degrade to LOCAL_ONE if needed; alert
Push (APNs/ FCM) outage	Offline users not woken	Vendor status page + delivery telemetry	Retry on schedule; users still get msg on next foreground open
Media Store / CDN outage	Media downloads fail; text still works	5xx rate, CDN dashboards	Fall back to origin region; show 'tap to retry' in UI
Auth/Key Server down	New conversations blocked (no prekey fetch)	X3DH error rate	Existing sessions keep working; degrade gracefully on first-contact only
Sender clock skew	Out-of-order display, wrong receipts	ts_client vs ts_server diff	Order by ts_server; UI displays best-effort ts_client
Replay / duplicate envelope	User sees same msg twice	msg_id collision	Idempotency: inbox_dedup table drops duplicate enqueue

WARNING

Degrade Gracefully

Order of preservation under stress: (1) text delivery, (2) receipts, (3) presence, (4) typing, (5) media. Presence and typing are the first to be silently dropped when the system is unhealthy — losing them is invisible to most users.

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Connection model	Persistent TLS per device	Battery + edge-server cost vs HTTP polling latency. At 1.16M msg/sec, polling is non-viable.
E2E encryption	Signal protocol everywhere	Compute + key-management cost; server can't help with search, spam ML on content. Privacy is the product.
Group fan-out	Sender-key (1× encrypt, server multicasts)	Rotation cost on member change vs O(N) per-message encrypt cost. Hybrid wins.
Server-side fan-out (plaintext)	REJECTED — incompatible with E2E	Would let server batch and dedup in plaintext, but breaks the privacy guarantee.
Storage model	Ephemeral relay (evict on ack)	Server can't be subpoenaed for history; users must back up themselves. WhatsApp's stance.
Inbox DB	Cassandra	Write-optimised, TTL native, ring-scalable. SQL would buckle at 1M writes/sec.
Media path	Side-channel object store + CDN	Two-step send (upload + envelope) vs inline. Inline would 50× the inbox volume.
Presence broadcast	Only to active-conversation contacts	Slight UI lag for stale chats vs 100× message storm.
Read receipts	Opt-out per user	Some senders never get the read tick; small ack savings; user trust win.
Push notifications	APNs / FCM (vendor lock)	Dependency on Apple/Google availability vs reinventing OS-level wakeup. Worth it.
Multi-device	Per-device keys, no server-held history	Linking a new device requires a primary; complex bootstrap vs trivial server-side sync.

INSIGHT

Headline Tension

WhatsApp is a system that aggressively gives up server-side capabilities — content search, spam ML on payload, server-side history — in exchange for the claim that the operator cannot read users' messages. Almost every architectural choice is downstream of that trade-off.

16

Interview Playbook

How to present this

WhatsApp is a classic 'chat at scale' interview. The interviewer wants to see you (a) do the capacity math, (b) reason about persistent connections, (c) understand fan-out vs storage trade-offs, and (d) speak intelligently about end-to-end encryption without hand-waving.

45-Minute Interview Outline

1. **Intro (2 min):** clarify scope — 1:1 + group, E2E, presence, attachments; voice/video out
2. **Capacity (5 min):** 100B msgs/day, 1.16M/sec avg, 100M concurrent conns, 100 PB/day media
3. **Connection model (5 min):** persistent TLS, ~50K/edge, heartbeats, GeoDNS
4. **High-level arch (3 min):** Edge → Router → {Inbox, Media, Presence, Push}
5. **Message lifecycle (5 min):** sent / delivered / read; envelope schema; ack chain
6. **Inbox model (5 min):** Cassandra per-device queue; eviction on ack; 30-day TTL cap
7. **E2E encryption (8 min):** X3DH initial handshake; Double Ratchet; sender keys for groups
8. **Group fan-out (4 min):** client-side vs server-side; why hybrid (sender key) wins
9. **Failures + trade-offs (5 min):** edge crash, Cassandra partition, server-as-relay choice
10. **Wrap (3 min):** what would change at 10× scale or with voice in scope

Key Talking Points

- “1.16M msgs/sec average, 5–10× peak” — shows you did the math
- “100M concurrent persistent connections, ~50K per edge → ~2K edge servers” — capacity → fleet sizing
- “Server is a relay, not an archive” — frames every other decision
- “X3DH for handshake, Double Ratchet for messages, sender keys for groups” — Signal in three phrases
- “Sender-key fan-out is $O(1)$ per message, $O(N)$ at setup” — quantifies the group trade-off
- “Media is a side-channel: ciphertext to S3, URL+key in the envelope” — explains the 100 PB/day in one sentence
- “Inbox row evicted on device ack” — privacy + storage in one move
- “Presence is broadcast only to active contacts” — avoids 50B presence msgs/day

Common Follow-ups

- **Q: Why not server-side fan-out for groups?** A: Server would need plaintext to dedup and re-encrypt — incompatible with E2E. Sender key gives $O(1)$ sender cost AND keeps server oblivious.

- **Q: How does WhatsApp claim it can't read messages?** A: Server only ever sees ciphertext + envelope metadata. Inbox row is ciphertext. Backups are encrypted client-side with user key. No keys on server.
- **Q: What if a recipient is offline for a year?** A: 30-day TTL on inbox row; after that, Cassandra evicts. Sender keeps single check. Retention is finite by design.
- **Q: How do you order messages?** A: ts_server assigned at sender-edge ingress is authoritative. ts_client is best-effort for UI. Per-pair monotonicity comes from the persistent TCP order, not a global clock.
- **Q: How do you handle a viral group flood?** A: Per-sender rate limit at the edge (cheap, before Cassandra). Group cap (~1024). Media is by-reference so storage doesn't explode.
- **Q: Why Cassandra instead of Kafka for the inbox?** A: Need point-lookups by recipient_device_id and per-row TTL/delete. Kafka is great for streams but not for selective replay/eviction.
- **Q: How do you scale presence?** A: Redis cluster sharded by user_id; broadcast only to subscribers with the chat open; heartbeat-derived; never persisted long-term.
- **Q: Multi-device — how does a new desktop get history?** A: Bootstrapped from a primary device over an authenticated channel; server doesn't hold the history.

KEY

Memorize These Numbers

2B users · 500M DAU · **100B msgs/day** · **1.16M msgs/sec avg**, ~10M peak · **100M concurrent connections** · ~50K/edge · ~2K edge servers · **100 PB/day media** · group cap ~1024 · inbox TTL 30 days · heartbeat 30–60 s.